Assignment 9: Time Series Analysis

Humayra Rahman

Spring 2026

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on generalized linear models.

Directions

1. Rename this file `<FirstLast>_A09_TimeSeries.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change "Student Name" on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.

Set up

1. Set up your session:

- Check your working directory
- Load the tidyverse, zoo, and trend packages
- Set your ggplot theme

```
# checking wd
getwd()
```

```
## [1] "/home/guest/EDE_Spring2026"
```

```
# loading packages
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(zoo)
```

```
##
## Attaching package: 'zoo'
##
## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
```

```r
library(trend)
library(here)
```

```
## here() starts at /home/guest/EDE_Spring2026
```

```r
library(Kendall)

# setting ggplot theme
mytheme <- theme_classic(base_size = 14) +
  theme(axis.text = element_text(color = "black"),
        legend.position = "top")
theme_set(mytheme)
```

2. Import the ten datasets from the Ozone_TimeSeries folder in the Raw data folder. These contain ozone concentrations at Garinger High School in North Carolina from 2010-2019 (the EPA air database only allows downloads for one year at a time). Import these either individually or in bulk and then combine them into a single dataframe named `GaringerOzone` of 3589 observation and 20 variables.

```r
#1 importing

GaringerOzone <- list.files(path = here("Data/Raw/Ozone_TimeSeries"),
                            pattern = "*.csv",
                            full.names = TRUE) %>%
  map_dfr(read.csv)

dim(GaringerOzone)
```

```
## [1] 3589   20
```

```r
# combing into single dataframe

GaringerOzone <- list.files(path = here("Data/Raw/Ozone_TimeSeries"),
                            pattern = "*.csv",
                            full.names = TRUE) %>%
  map_dfr(~ read.csv(.x, stringsAsFactors = TRUE))

dim(GaringerOzone)
```

```
## [1] 3589   20
```

Wrangle

3. Set your date column as a date class.
4. Wrangle your dataset so that it only contains the columns Date, Daily.Max.8.hour.Ozone.Concentration, and DAILY_AQI_VALUE.
5. Notice there are a few days in each year that are missing ozone concentrations. We want to generate a daily dataset, so we will need to fill in any missing days with NA. Create a new data frame that contains a sequence of dates from 2010-01-01 to 2019-12-31 (hint: `as.data.frame(seq())`). Call this new data frame Days. Rename the column name in Days to "Date".
6. Use a `left_join` to combine the data frames. Specify the correct order of data frames within this function so that the final dimensions are 3652 rows and 3 columns. Call your combined data frame GaringerOzone.

```r
# 3 setting date colm as date class
GaringerOzone$Date <- as.Date(GaringerOzone$Date, format = "%m/%d/%Y")

class(GaringerOzone$Date)
```

```
## [1] "Date"
```

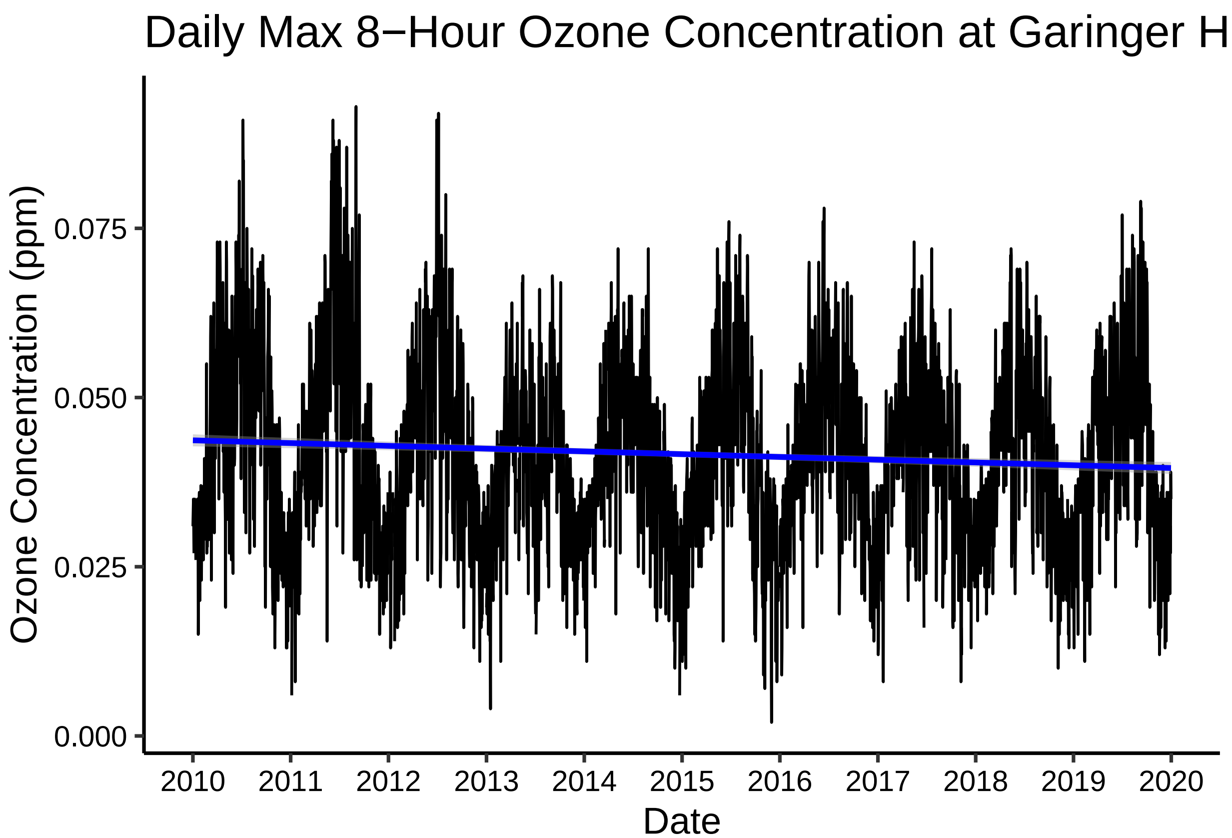
```r
# 4 wrangling dataset
GaringerOzone <- GaringerOzone %>%
  select(Date, Daily.Max.8.hour.Ozone.Concentration, DAILY_AQI_VALUE)

# 5

# creating a dataframe with every date from 2010 to 2019
Days <- as.data.frame(seq(as.Date("2010-01-01"),
                          as.Date("2019-12-31"), by = "day"))

# renaming the column to "Date"
colnames(Days) <- "Date"

# 6 combing dataframes
GaringerOzone <- left_join(Days, GaringerOzone, by = "Date")
```

Visualize

7. Create a line plot depicting ozone concentrations over time. In this case, we will plot actual concentrations in ppm, not AQI values. Format your axes accordingly. Add a smoothed line showing any linear trend of your data. Does your plot suggest a trend in ozone concentration over time?

```r
#7 creating line plot

ggplot(GaringerOzone, aes(x = Date, y = Daily.Max.8.hour.Ozone.Concentration)) +
  geom_line() +
  geom_smooth(method = "lm", color = "blue") +
  labs(x = "Date",
       y = "Ozone Concentration (ppm)",
       title = "Daily Max 8-Hour Ozone Concentration at Garinger High School (2010-2019)") +
  scale_x_date(date_breaks = "1 year", date_labels = "%Y")
```

```
## 'geom_smooth()' using formula = 'y ~ x'

## Warning: Removed 63 rows containing non-finite outside the scale range
## ('stat_smooth()').
```



Answer: The plot shows a very slight downward trend in ozone concentrations between 2010 and 2019. While the linear trend line does slope downward, it is nearly flat, suggesting any decline is minimal. What stands out more is the strong seasonal pattern where concentrations rise and fall predictably each year, peaking in the warmer months. Given how subtle the trend is, it is hard to say anything definitive just from looking at the plot, so statistical testing would be needed to confirm whether the decrease is real and significant.

Time Series Analysis

Study question: Have ozone concentrations changed over the 2010s at this station?

8. Use a linear interpolation to fill in missing daily data for ozone concentration. Why didn't we use a piecewise constant or spline interpolation?

```r
#8 linear interpolation

GaringerOzone$Daily.Max.8.hour.Ozone.Concentration <-
  na.approx(GaringerOzone$Daily.Max.8.hour.Ozone.Concentration)
```

Answer: Linear interpolation makes the most sense here because ozone concentrations change gradually, so estimating a missing day as a value between its neighbors is reasonable. Piecewise constant interpolation was avoided because simply repeating the last known value is unrealistic for a continuously changing variable. Spline interpolation, while smoother, is overly complex for this situation and can produce unrealistic values when the surrounding data is highly variable.

9. Create a new data frame called `GaringerOzone.monthly` that contains aggregated data: mean ozone concentrations for each month. In your pipe, you will need to first add columns for year and month to form the groupings. In a separate line of code, create a new Date column with each month-year combination being set as the first day of the month (this is for graphing purposes only)

```
#9
# creating monthly aggregated dataframe
GaringerOzone.monthly <- GaringerOzone %>%
  mutate(Month = month(Date),
         Year = year(Date)) %>%
  group_by(Year, Month) %>%
  summarise(Mean.Ozone = mean(Daily.Max.8.hour.Ozone.Concentration))
```

```
## `summarise()` has grouped output by 'Year'. You can override using the
## `.groups` argument.
```

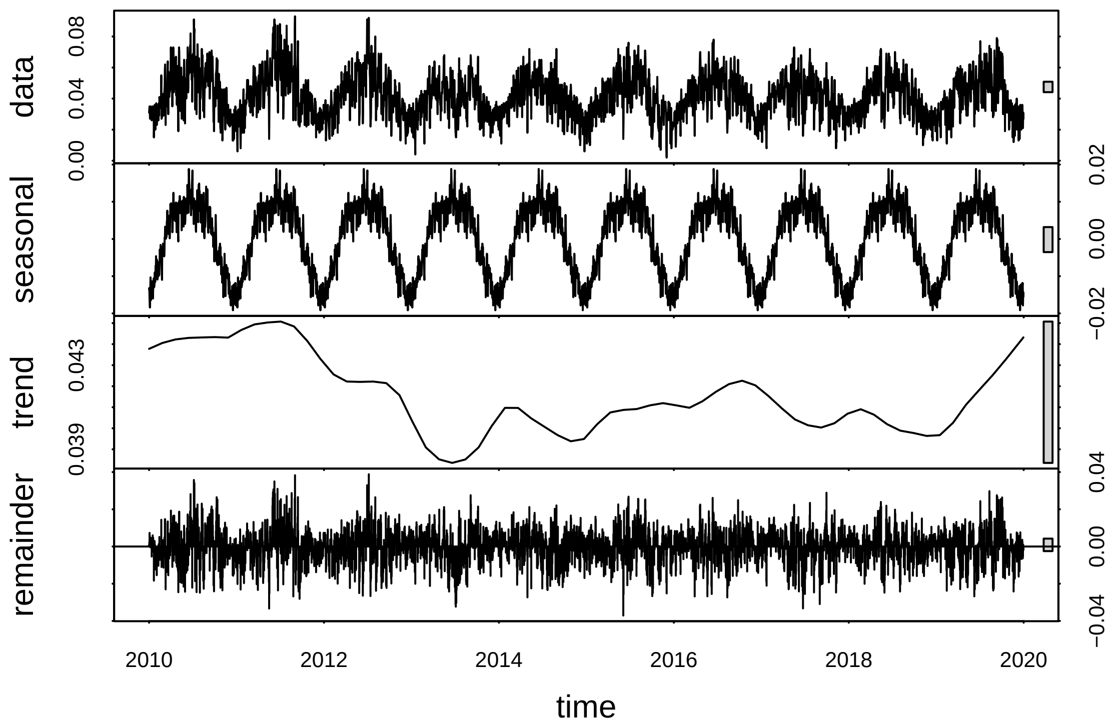
```
# creating new date coln set to first day of each month
GaringerOzone.monthly$Date <- as.Date(paste(GaringerOzone.monthly$Year,
                                            GaringerOzone.monthly$Month,
                                            "1", sep = "-"))
```

10. Generate two time series objects. Name the first `GaringerOzone.daily.ts` and base it on the dataframe of daily observations. Name the second `GaringerOzone.monthly.ts` and base it on the monthly average ozone values. Be sure that each specifies the correct start and end dates and the frequency of the time series.

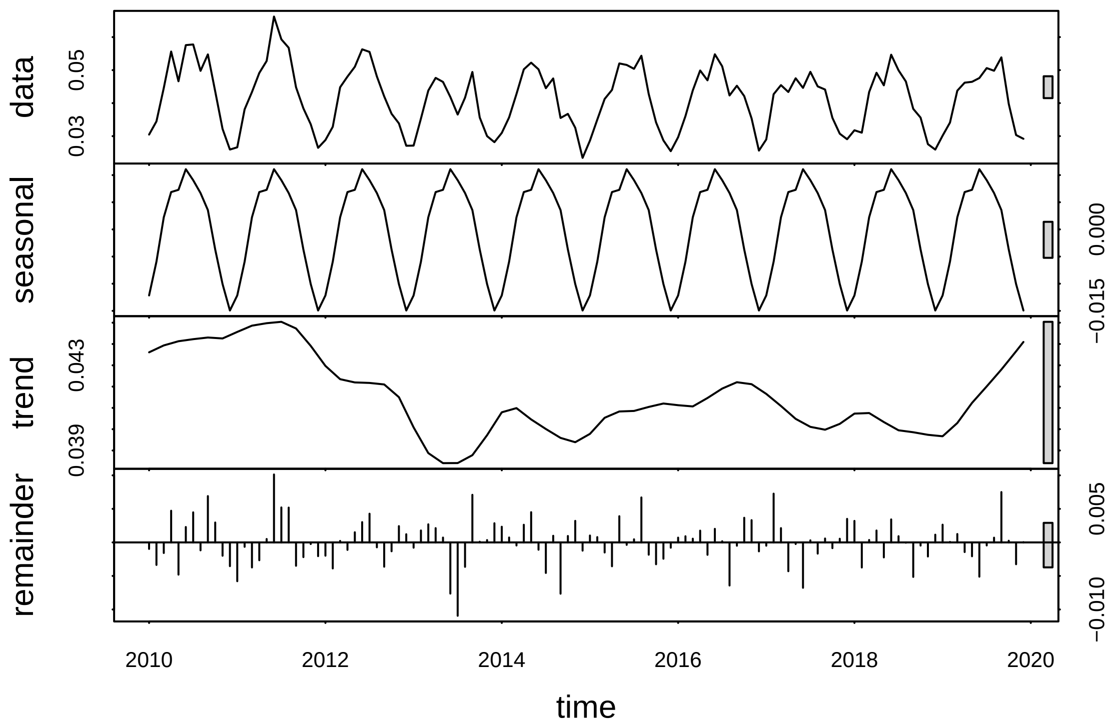
```
#10

# daily time series
GaringerOzone.daily.ts <- ts(GaringerOzone$Daily.Max.8.hour.Ozone.Concentration,
                             start = c(2010, 1),
                             end = c(2019, 365),
                             frequency = 365)

# monthly time series
GaringerOzone.monthly.ts <- ts(GaringerOzone.monthly$Mean.Ozone,
                               start = c(2010, 1),
                               end = c(2019, 12),
                               frequency = 12)
```

11. Decompose the daily and the monthly time series objects and plot the components using the `plot()` function.

```r
#11

# decomposing daily time series
GaringerOzone.daily.decomposed <- stl(GaringerOzone.daily.ts, s.window = "periodic")
plot(GaringerOzone.daily.decomposed)
```




```r
# decomposing monthly time series
GaringerOzone.monthly.decomposed <- stl(GaringerOzone.monthly.ts, s.window = "periodic")
plot(GaringerOzone.monthly.decomposed)
```



12. Run a monotonic trend analysis for the monthly Ozone series. In this case the seasonal Mann-Kendall is most appropriate; why is this?

```r
#12

# running seasonal Mann-Kendall test on monthly time series
GaringerOzone.monthly.trend <- trend::smk.test(GaringerOzone.monthly.ts)

# inspecting results
GaringerOzone.monthly.trend
```

```
##
##  Seasonal Mann-Kendall trend test (Hirsch-Slack test)
##
## data:  GaringerOzone.monthly.ts
## z = -1.963, p-value = 0.04965
## alternative hypothesis: true S is not equal to 0
## sample estimates:
##    S varS
##  -77 1499
```

```r
summary(GaringerOzone.monthly.trend)
```

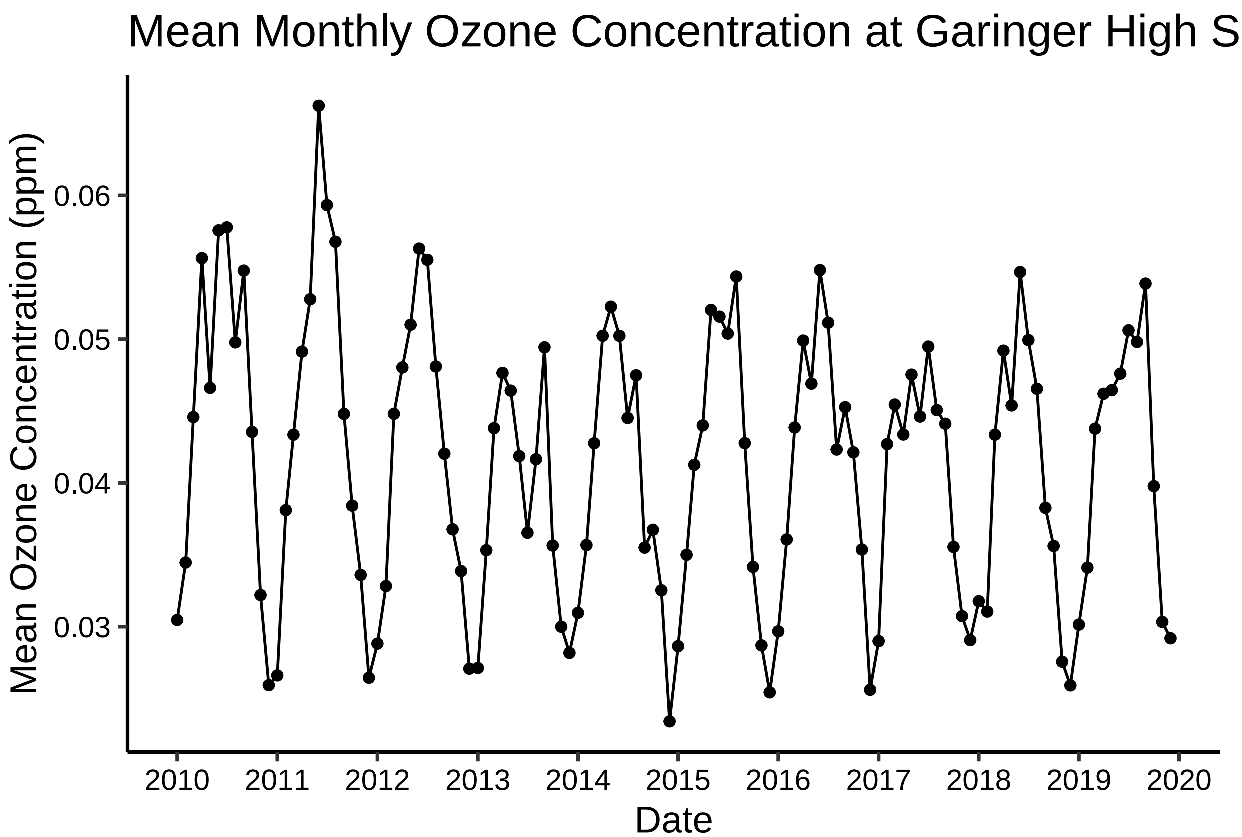
```
##
```

```
##  Seasonal Mann-Kendall trend test (Hirsch-Slack test)
##
## data: GaringerOzone.monthly.ts
## alternative hypothesis: two.sided
##
## Statistics for individual seasons
##
## H0
##                         S  varS    tau      z Pr(>|z|)
## Season 1:   S = 0   15  125  0.333  1.252  0.21050
## Season 2:   S = 0   -1  125 -0.022  0.000  1.00000
## Season 3:   S = 0   -4  124 -0.090 -0.269  0.78762
## Season 4:   S = 0  -17  125 -0.378 -1.431  0.15241
## Season 5:   S = 0  -15  125 -0.333 -1.252  0.21050
## Season 6:   S = 0  -17  125 -0.378 -1.431  0.15241
## Season 7:   S = 0  -11  125 -0.244 -0.894  0.37109
## Season 8:   S = 0   -7  125 -0.156 -0.537  0.59151
## Season 9:   S = 0   -5  125 -0.111 -0.358  0.72051
## Season 10:   S = 0 -13  125 -0.289 -1.073  0.28313
## Season 11:   S = 0 -13  125 -0.289 -1.073  0.28313
## Season 12:   S = 0  11  125  0.244  0.894  0.37109
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Answer: The seasonal Mann-Kendall is most appropriate here because the ozone data has a clear seasonal component. Standard tests like the regular Mann-Kendall or linear regression assume no seasonality, so applying them directly would confound the results. The seasonal Mann-Kendall gets around this by comparing each month to the same month across all years, effectively removing the seasonal influence and isolating the true long term trend.

13. Create a plot depicting mean monthly ozone concentrations over time, with both a geom_point and a geom_line layer. Edit your axis labels accordingly.

```r
# 13 creating plot

ggplot(GaringerOzone.monthly, aes(x = Date, y = Mean.Ozone)) +
  geom_point() +
  geom_line() +
  labs(x = "Date",
       y = "Mean Ozone Concentration (ppm)",
       title = "Mean Monthly Ozone Concentration at Garinger High School (2010-2019)") +
  scale_x_date(date_breaks = "1 year", date_labels = "%Y")
```



14. To accompany your graph, summarize your results in context of the research question. Include output from the statistical test in parentheses at the end of your sentence. Feel free to use multiple sentences in your interpretation.

Answer: Mean monthly ozone concentrations at Garinger High School declined significantly over the 2010 to 2019 period. While a clear seasonal cycle is visible each year, with concentrations peaking in warmer months, the overall trajectory is downward. Interestingly, no individual month drives this trend on its own, suggesting the decline is gradual and spread across all seasons (Seasonal Mann-Kendall; z = -1.963, S = -77, p = 0.04965).

15. Subtract the seasonal component from the `GaringerOzone.monthly.ts`. Hint: Look at how we extracted the series components for the EnoDischarge on the lesson Rmd file.

16. Run the Mann Kendall test on the non-seasonal Ozone monthly series. Compare the results with the ones obtained with the Seasonal Mann Kendall on the complete series.

```r
#15

# extracting the seasonal component and subtract from monthly ts
GaringerOzone.monthly.components <- as.data.frame(GaringerOzone.monthly.decomposed$time.series[,1:3])

GaringerOzone.monthly.nonseasonal <- GaringerOzone.monthly.ts -
  GaringerOzone.monthly.decomposed$time.series[,1]
```

```r
#16

# running Mann-Kendall test on non-seasonal series
GaringerOzone.monthly.nonseasonal.trend <- MannKendall(GaringerOzone.monthly.nonseasonal)

# inspecting results
GaringerOzone.monthly.nonseasonal.trend
```

```
## tau = -0.165, 2-sided pvalue =0.0075402
```

```r
summary(GaringerOzone.monthly.nonseasonal.trend)
```

```
## Score =  -1179 , Var(Score) = 194365.7
## denominator =  7139.5
## tau = -0.165, 2-sided pvalue =0.0075402
```

> Answer: Both tests confirm a downward trend in ozone concentrations over the 2010 to 2019 period, but the Mann-Kendall on the de-seasonalized series gave a stronger result (p = 0.0075) compared to the Seasonal Mann-Kendall on the complete series (p = 0.04965). This is expected since removing the seasonal component reduces noise, making the underlying trend easier to detect. Either way, the conclusion is the same: ozone concentrations at Garinger High School have declined over the decade (Mann-Kendall; tau = -0.165, p = 0.0075).